



L3 informatique

Course Notes
Software Engineering

Course 3
Sequence diagram

Presented by
Mr KHELIFA N.

Interaction diagram

- Shows an interaction = A set of objects and their relationships, as well as the messages that can circulate between them.
- Interaction diagrams exist in different variants:
 - Sequence diagram
 - Communication diagram
 - Interaction overview diagram
 - Timing diagram

Sequence diagram

- The purpose of the sequence diagram is to represent the interactions between objects (system objects or actor objects) while indicating the chronology of exchanges. The sequence diagram highlights the ordering of messages in chronological order.
- It is used:
 - During requirements capture (system = black box):**
 - Interactions between actors and the system
 - Describing use case scenarios
 - During design (system = white box):**
 - Interactions between objects
 - Considering the assignment of responsibilities to objects
 - Who creates the objects?
 - Who provides access to an object?

Exemple

- A driver starts a car, and the car turns on an engine.

Exemple

- A driver starts a car, and the car turns on an engine.

Objects

- A driver starts a car, and the car turns on an engine.

Example

- A driver starts a car, and the car turns on an engine.

Objects

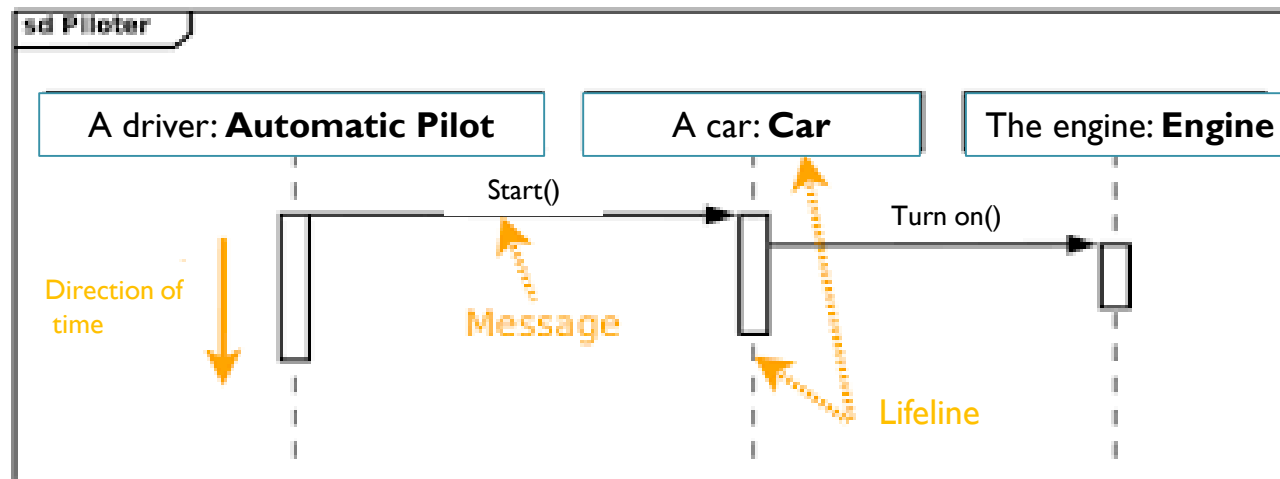
- A driver starts a car, and the car turns on an engine.

Message

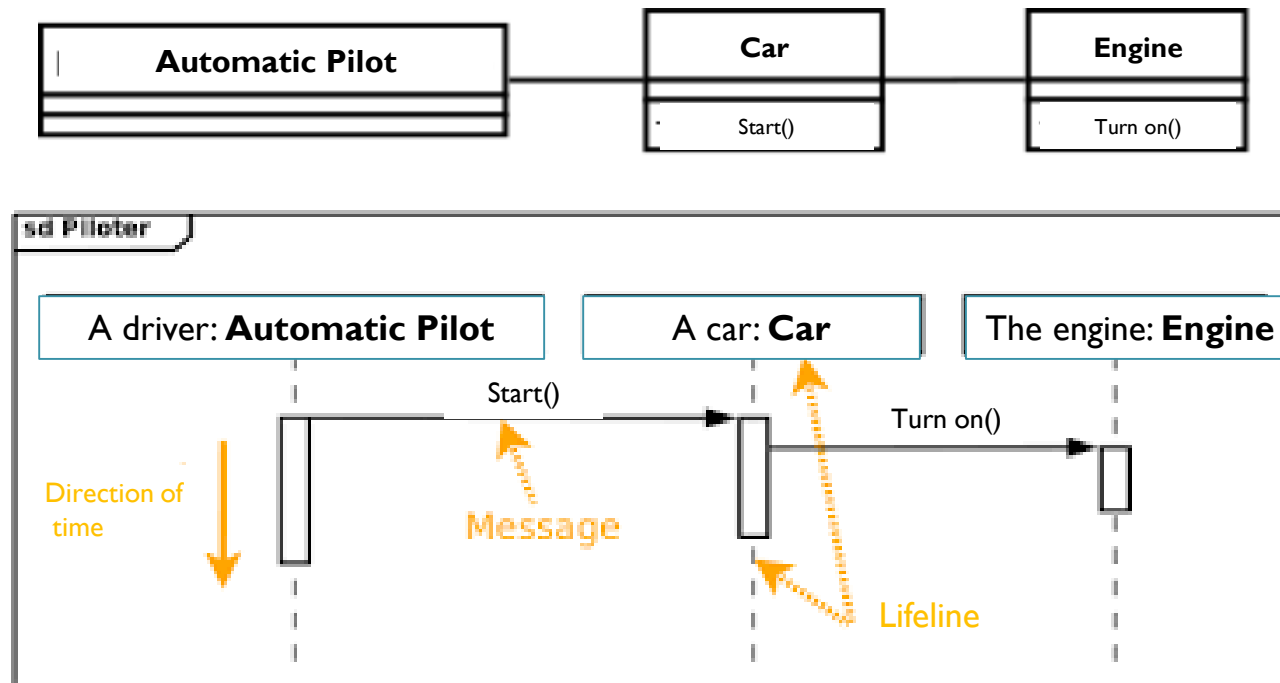
- A driver starts a car, and the car turns on an engine.

Example

- A driver starts a car, and the car turns on an engine.

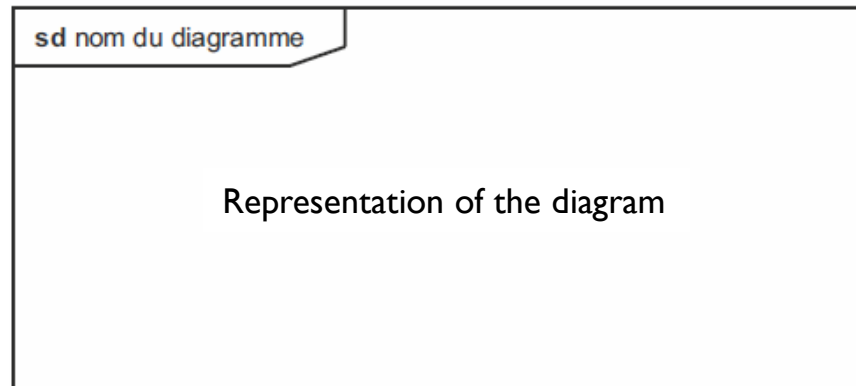


Example



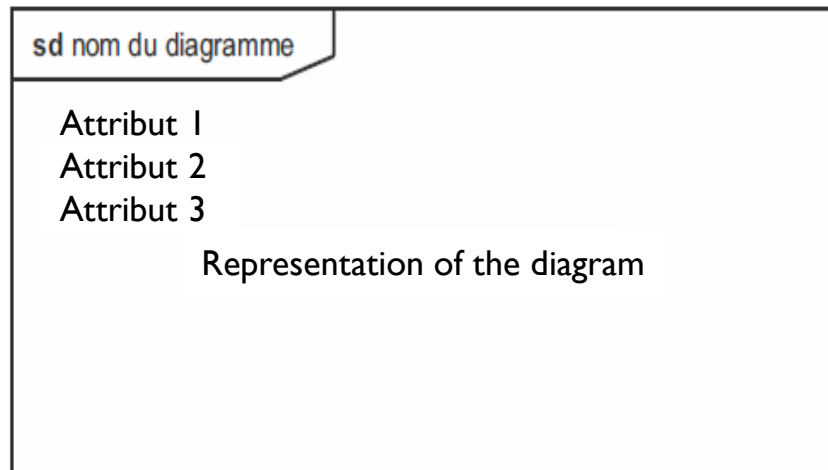
Sequence diagram (notation)

- The sequence diagram is placed inside a rectangle that has a “**sd**” label at the top left (which stands for *sequence diagram*), followed by the name of the diagram.



Local attribute

- Definitions of local attributes use the same syntax as general attributes displayed in the compartments of class symbols.
- These attribute definitions can appear at the top of the diagram frame or in note symbols at other locations within the diagram.
- The way attributes are written is identical to that used in class diagrams.

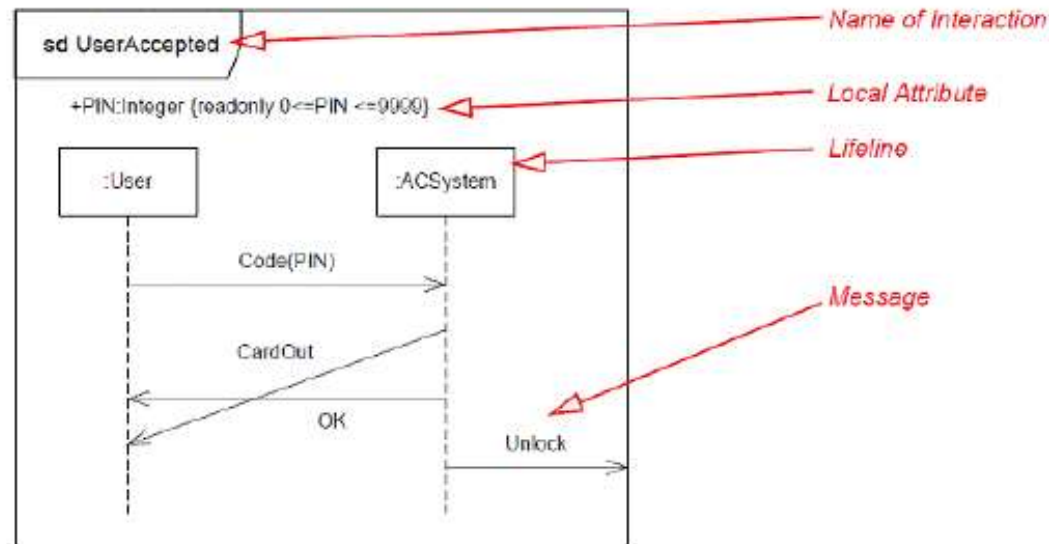


Exemple

- During authentication with an **ACMSsystem**, the user enters the **PIN code**.
This code is of **integer** type, **read-only**, and its value is limited between **0** and **99999**.

Example

- During authentication with an **ACMS**ystem, the user enters the **PIN** code.
This code is of **integer** type, **read-only**, and its value is limited between **0** and **99999**.



Sequence diagram

- The sequence diagram includes:
 - **Lifeline** (object + vertical line)
 - **Messages**
 - **Combined fragment**

The object and the actor

- In a sequence diagram, the object has the same representation as in the object diagram.
- The object is represented by a rectangle containing the object's name.
- The object's name is **underlined** and can take one of the following four forms:

nomObjet : NomClasse

roleObjet :Nom Classe

nomObjet :

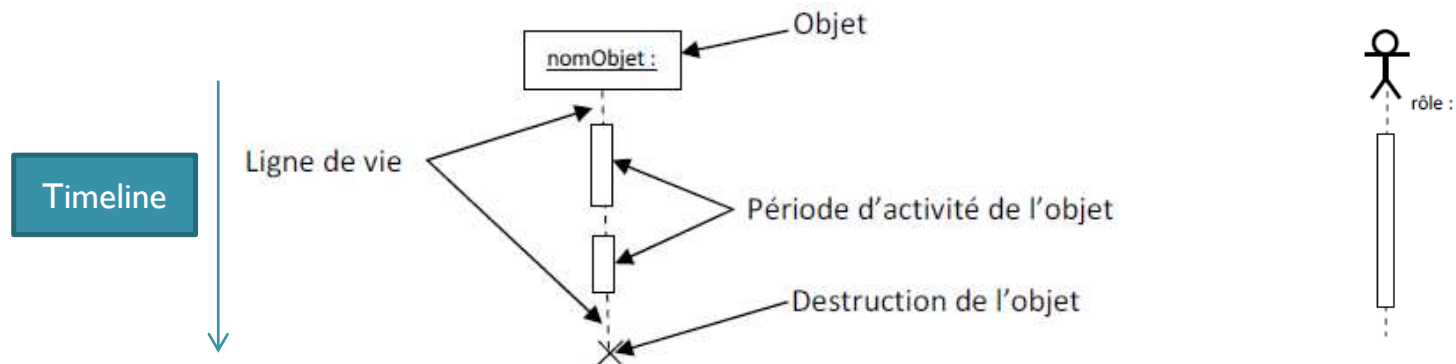
: NomClasse



Rôle :

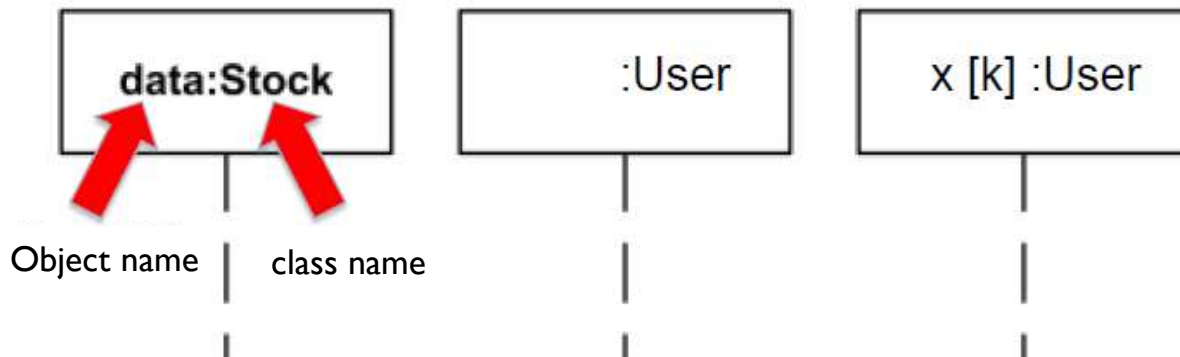
Lifeline

- A lifeline represents a participant in an interaction (object or actor).
- Each object is associated with a vertical line (a dashed line drawn vertically below the object), which can be considered a time axis (time flows from top to bottom).
- In this type of diagram, the quantification of time is not important.
- The lifeline indicates the periods of activity of the object — these activity periods generally correspond to the moments when the object executes one of its methods.
- When the object is destroyed, the lifeline ends with a cross.



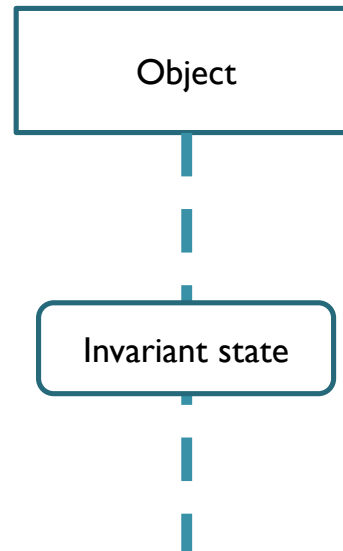
Timeline

- Notation:
Box representing the object
lifelineName {[selector]} : **ClassOrActorName**
Vertical dashed line



StateInvariant

- The state is said to be invariant when it remains constant along a lifeline. It may correspond to the internal state of the classifier, or to an external state — a state based on a “black-box” view of the lifeline.

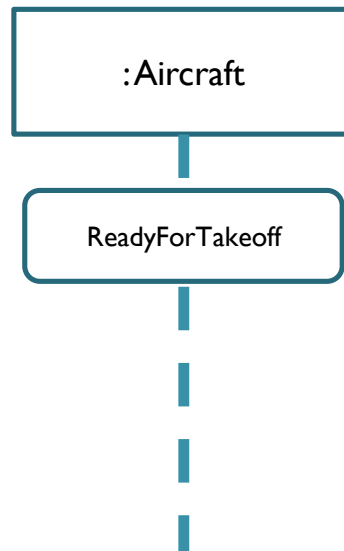


Exemple

- Regardless of the context, at the moment of takeoff the aircraft must be in the state “ReadyForTakeoff”.

Exemple

- Regardless of the context, at the moment of takeoff the aircraft must be in the state “ReadyForTakeoff”.



Message

- **Message:**

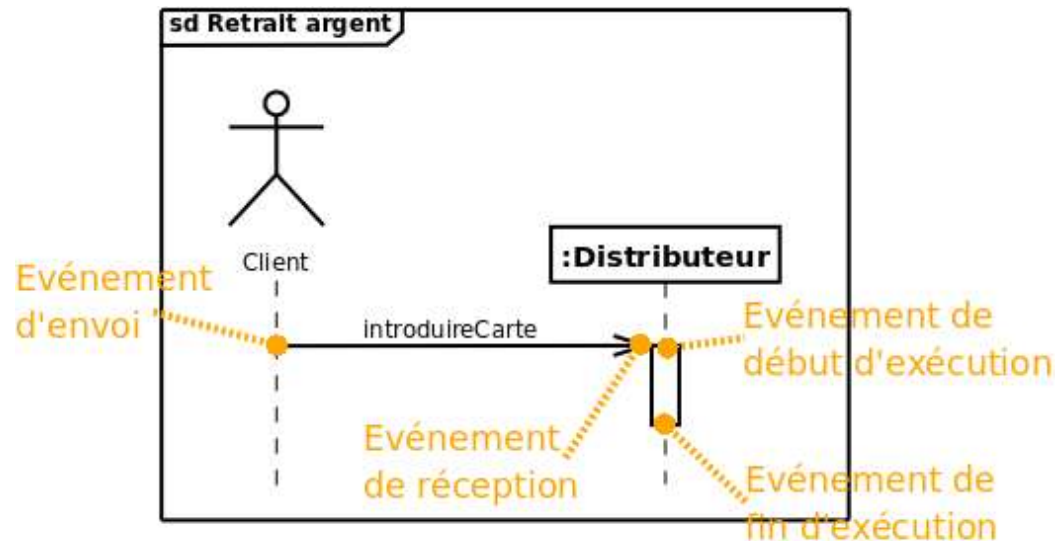
A message defines a specific communication between lifelines.

A message is a communication from one object to another.

The reception of a message is considered by the receiving object as an event that it may handle (or not).

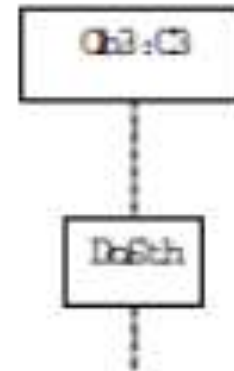
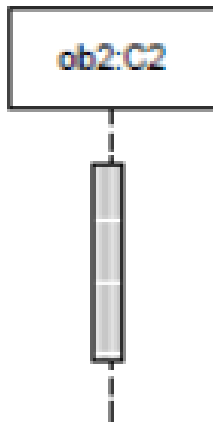
- A message represents:

- The call of an operation (method) and the start of its execution
- Sending and receiving a signal



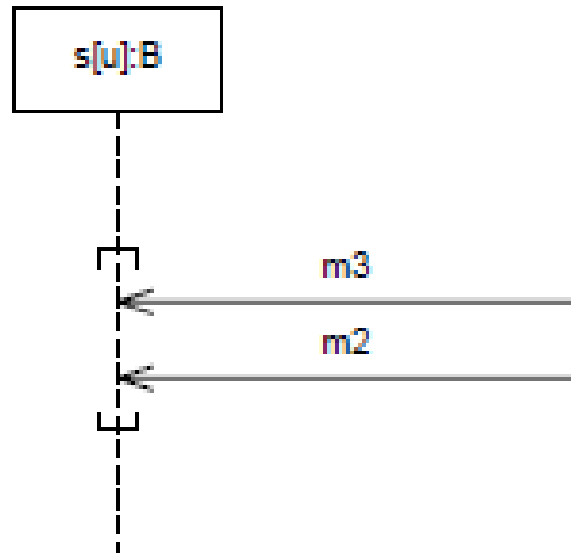
ExecutionSpecification

- The trace semantics of Interactions merely see an Execution as the trace <start, finish>
- ExecutionSpecifications are represented as
 - thin rectangles (gray or white) on the lifeline
 - by a wider labeled rectangle, where the label usually identifies the action that was executed.



coregion

- In a coregion, the order of events does not matter:
the messages or actions within it may occur at the same time or in any order.

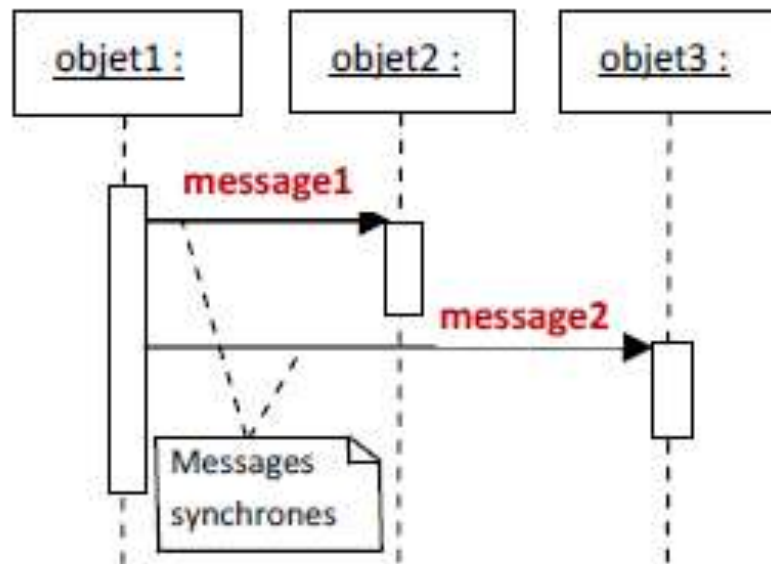


Types of messages

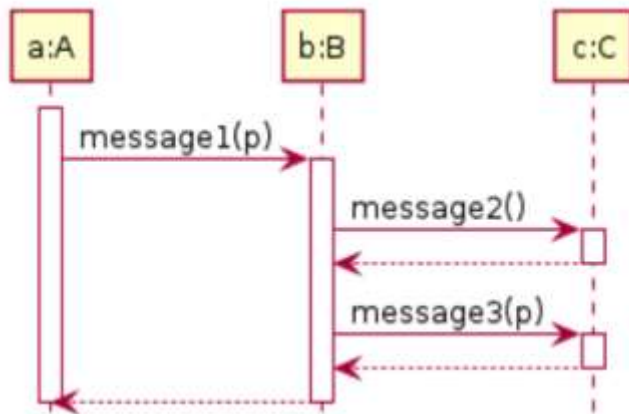
- A message may be:
 - A synchronous call
 - An asynchronous call
 - A creation
 - A destruction
 - A return (response)

Synchronous messages

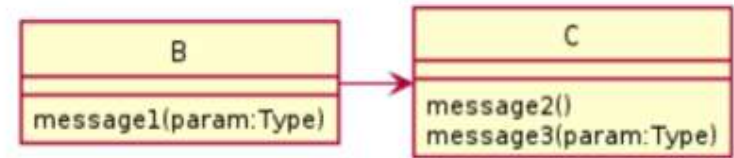
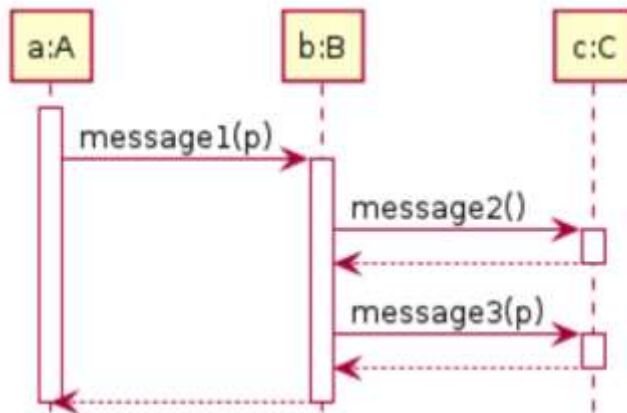
- Represents a method call
The sender sends a message
It suspends its execution while waiting for the response
Solid arrowhead



Synchronous messages and class diagram

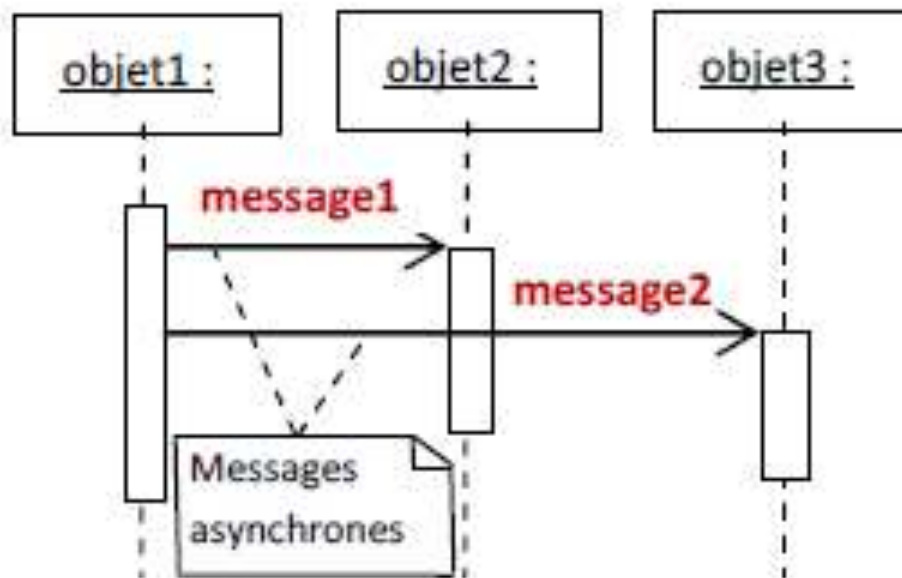


Synchronous messages and class diagram



Asynchronous call

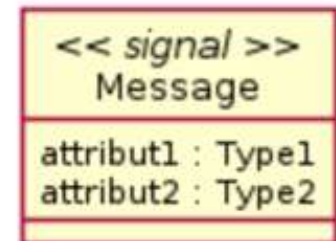
- Represents a method call
 - The sender sends a message
 - It continues its execution without waiting for the response
- Open arrowhead



Asynchronous messages and class diagram



Asynchronous messages and class diagram



Exemple

- An ATM checks a client's balance with the bank before performing a transaction.

An elevator turns on an indicator light when passing each floor.

Exemple

- An ATM checks a client's balance with the bank before performing a transaction.



The ATM remains blocked while waiting for the response.

An elevator turns on an indicator light when passing each floor.



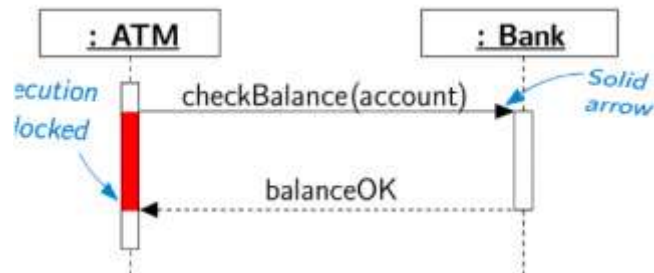
An elevator remains unblocked and continues its execution

Exemple

- An ATM checks a client's balance with the bank before performing a transaction.



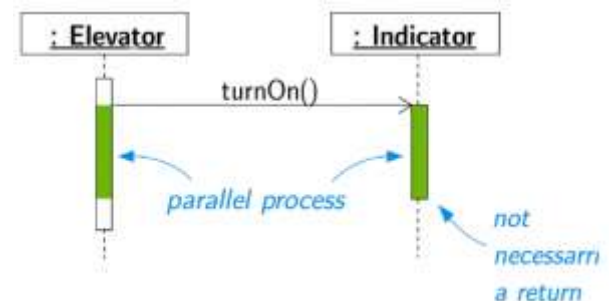
The ATM remains blocked while waiting for the response.



An elevator turns on an indicator light when passing each floor.



An elevator remains unblocked and continues its execution



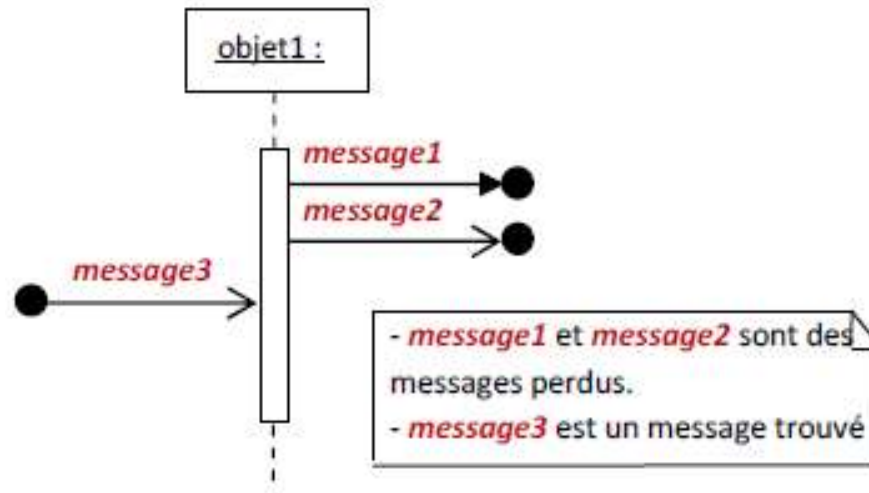
Lost and found message

A **lost message** is one for which the send event is known, but the receive event is not.

It is represented by an arrow pointing to a small black dot.

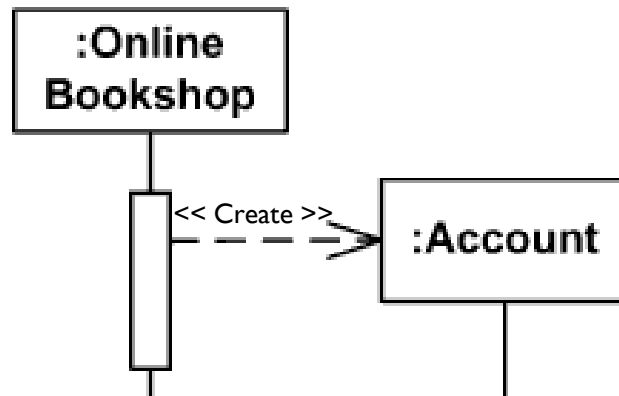
A **found message** is one for which the receive event is known, but the send event is not.

An arrow starting from a small black dot represents a found message.



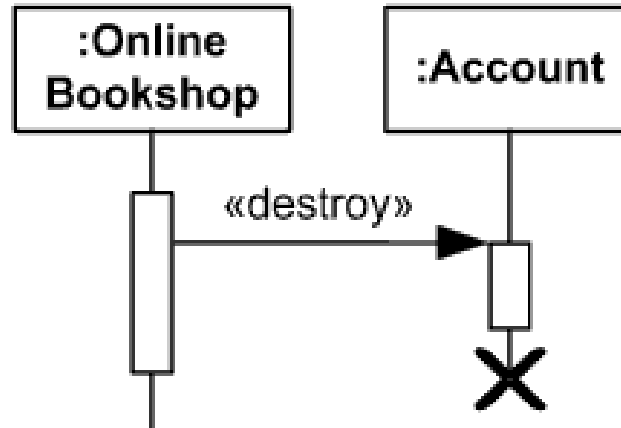
Creation Message

- The creation of an object is represented by a specific message, a constructor call, usually accompanied by the « create » stereotype, which points to the beginning (the top) of the object's lifeline (the instance's rectangle is then lowered). Dashed arrow, open at its end, pointing to the top of a lifeline.



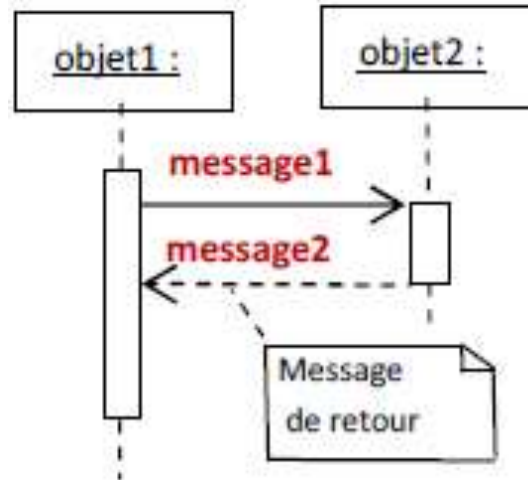
Destruction Message

- Message sent to a lifeline to terminate it. The lifeline ends with a cross.



Destruction Message (reponse)

- If a method that has been activated (by a message) must return values at the end of its activation, this is done through a **return message**. The return message is therefore **not a method call** (it does not trigger the activation of an object). The return message often carries the name of the returned element. Dashed arrow with an open arrowhead.



Syntaxe des messages synchrones et asynchrones

- In most cases, receiving a message is followed by the execution of the method of the target class. This method may receive arguments, and the message syntax allows these arguments to be passed.
- Most of the time, in a sequence diagram, we can define a message simply by:
- Its name (which is the name of the called method or the sent signal),
*: indicates any message.
- We can optionally add:
- A numbering before the message name (separated from the message name by a colon ":"). Numbering is done sequentially starting from 1.
- The parameters passed to the method or signal (in parentheses after the message name).

Numérotation :

nomMessage

(Paramètres)

Syntax of return messages

- As with synchronous or asynchronous messages, we can simply give the return message a name, but we can also describe it more precisely using the following syntax:

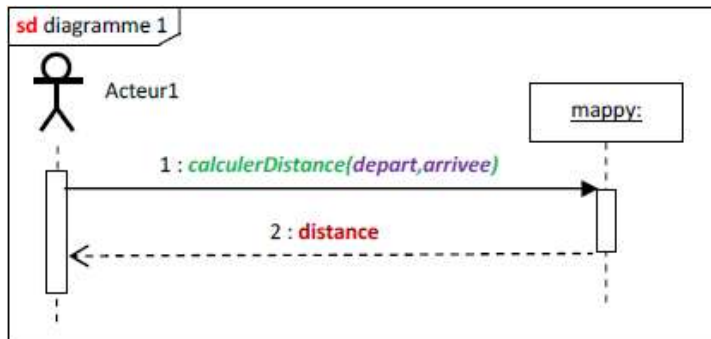
Numero :	attribut=	nomMessage	(Paramètres)	:valeurDeRetour
----------	-----------	------------	--------------	-----------------

Exemple

Actor I calculates the distance 'between departure and arrival'; the system returns the distance according to the cases.

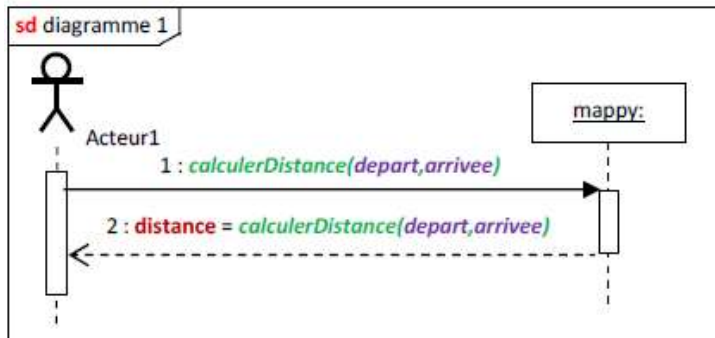
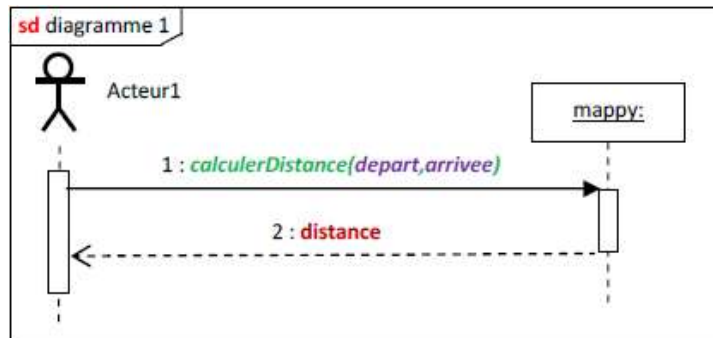
Exemple

Actor 1 calculates the distance 'between departure and arrival'; the system returns the distance according to the cases.



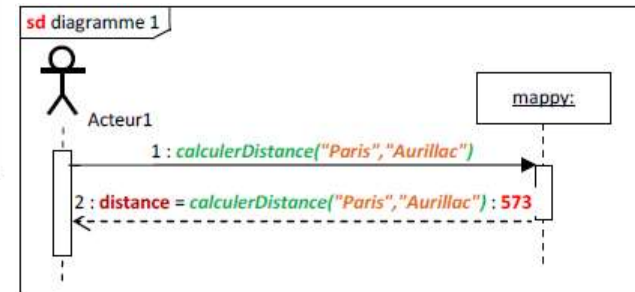
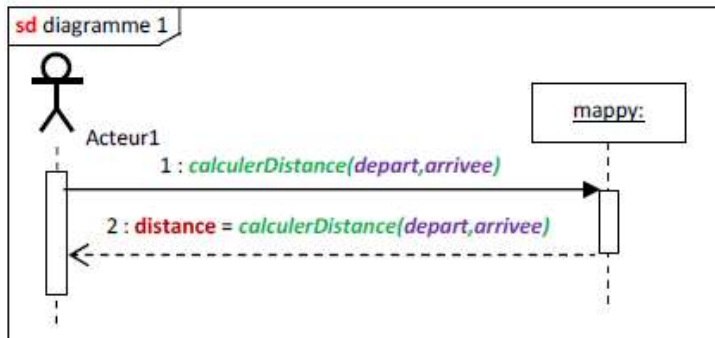
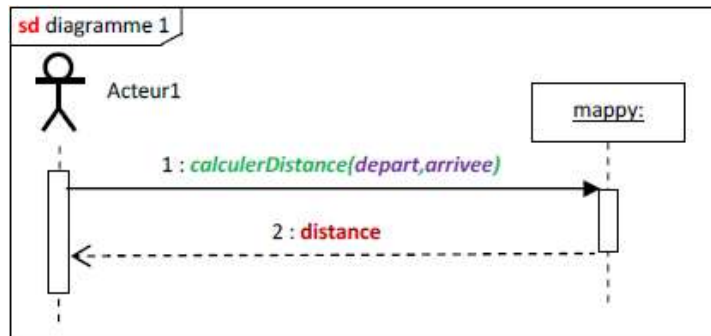
Exemple

Actor I calculates the distance 'between departure and arrival'; the system returns the distance according to the cases.



Exemple

Actor I calculates the distance 'between departure and arrival'; the system returns the distance according to the cases.

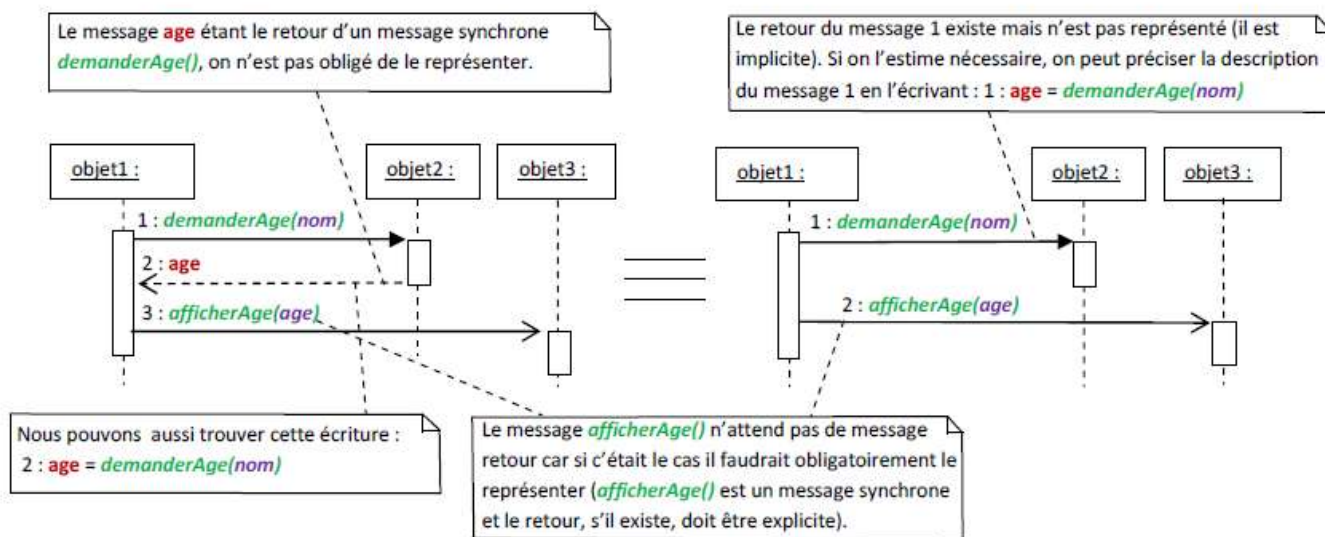


Exemple

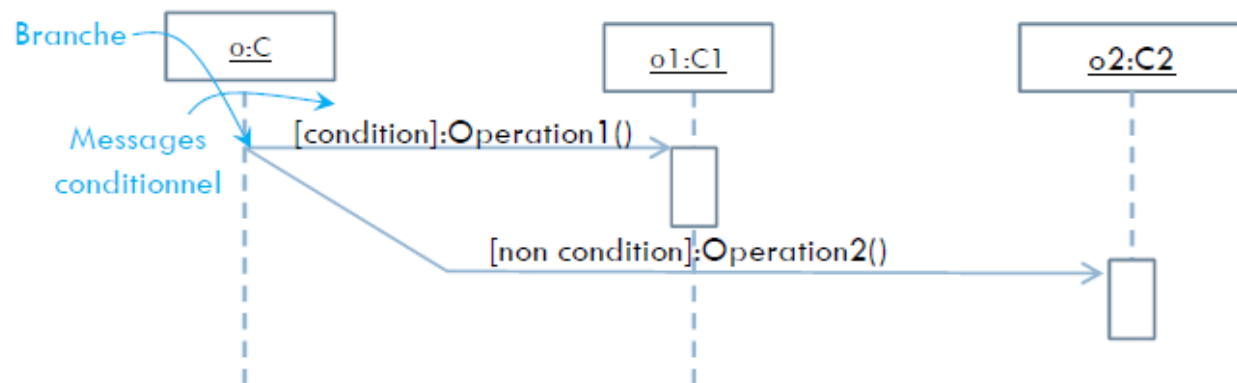
Object 1 requests the age from Object 2 by passing the parameter name, then displays the age in Object 3."

Exemple

Object 1 requests the age from Object 2 by passing the parameter name, then displays the age in Object 3."

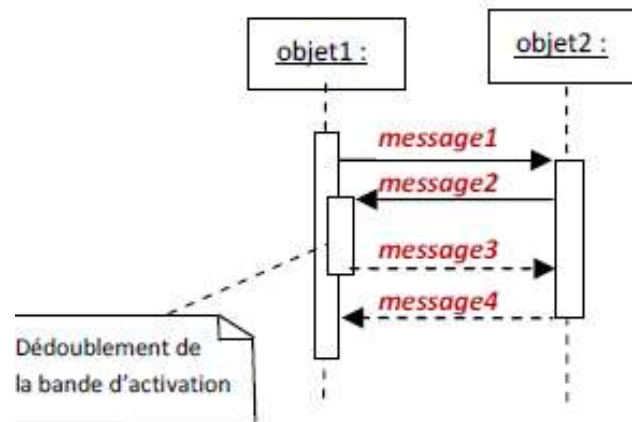


Message conditionnel

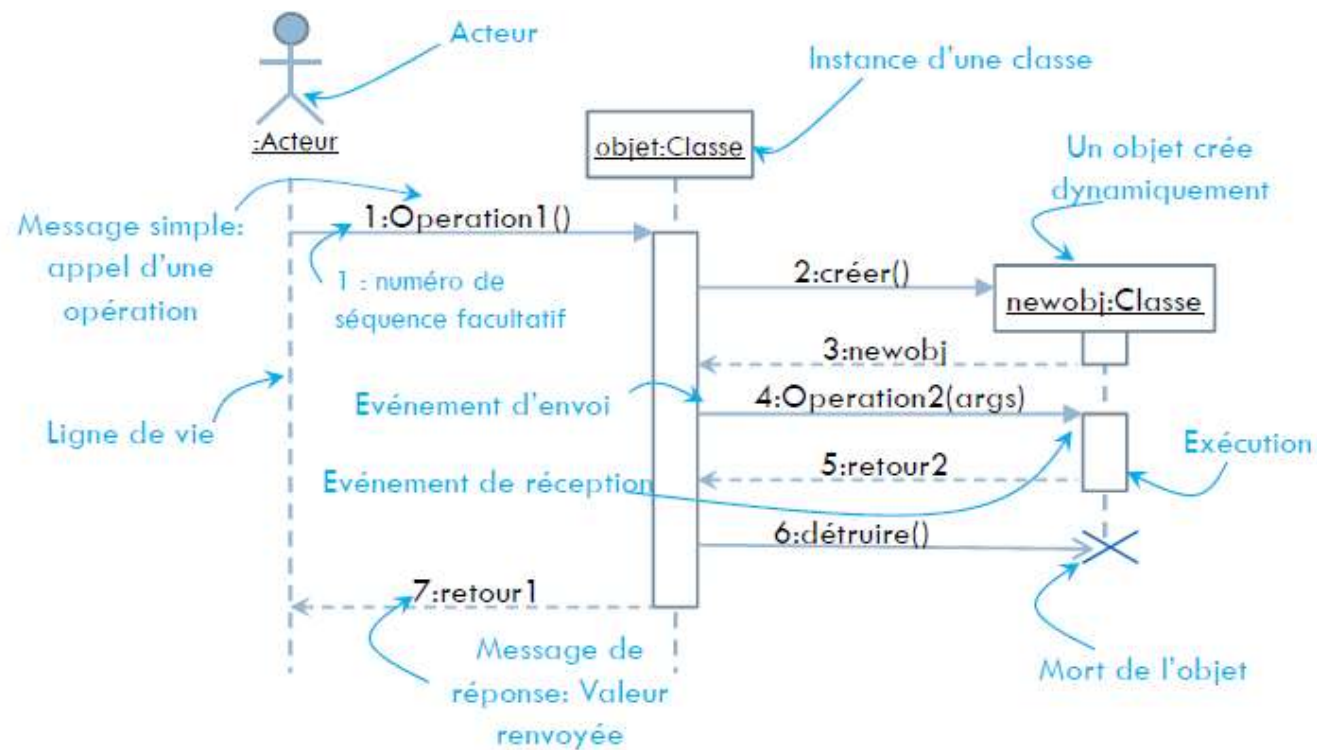


Overlap of activation bands

- **Overlapping activation bars:**
When an object is already activated, it can still receive other messages (calling another one of its methods). This is represented by a duplication (overlapping) of the activation bar.



Exemple

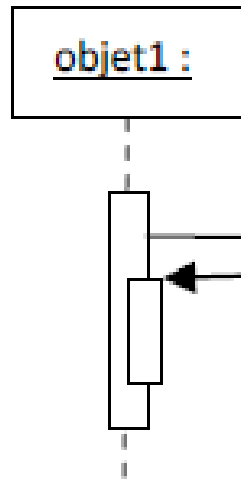


Recursive messages

- **Recursive messages:**

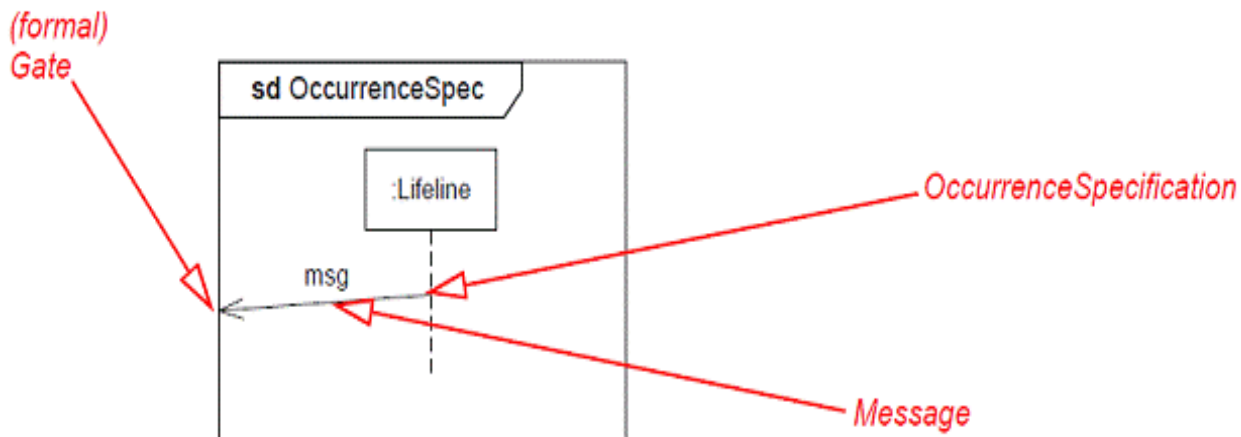
An object can send a message to itself (by using another method of the same object).

This is also represented by a splitting of the activation bar."

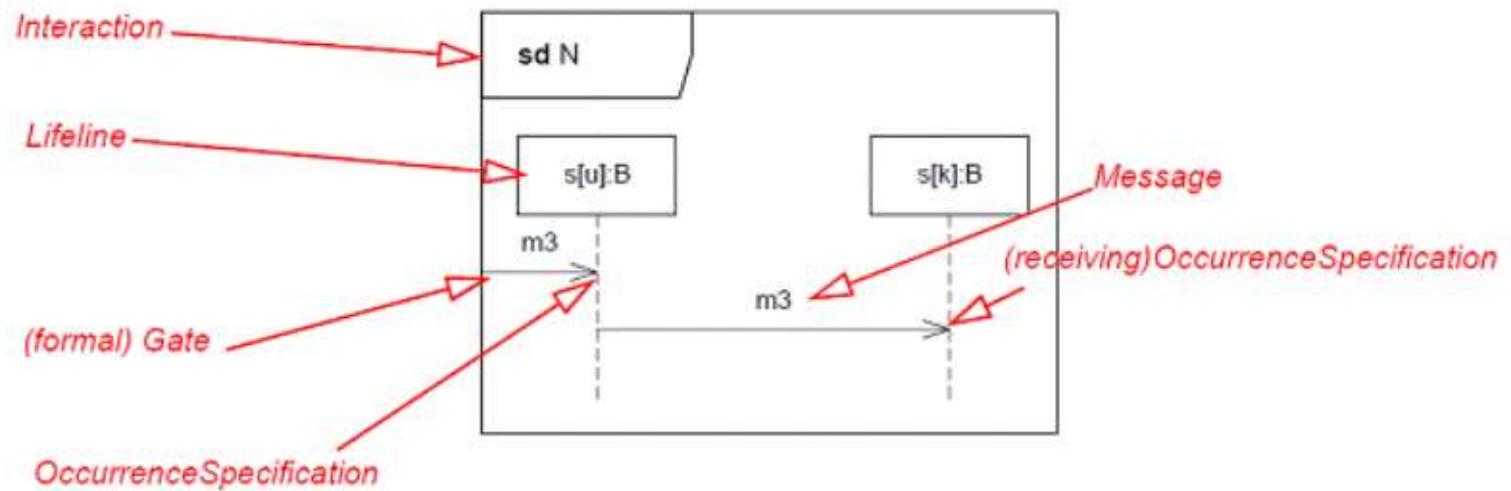


Gates

- **Gates** are simply points on the frame (the boundary of an interaction, interaction use, or combined fragment), representing the endpoints of messages.
- An entry is a connection point of a message inside a fragment with a message outside a connection fragment. It represents the source of incoming messages or the target of outgoing messages.
- They may have an explicit name.

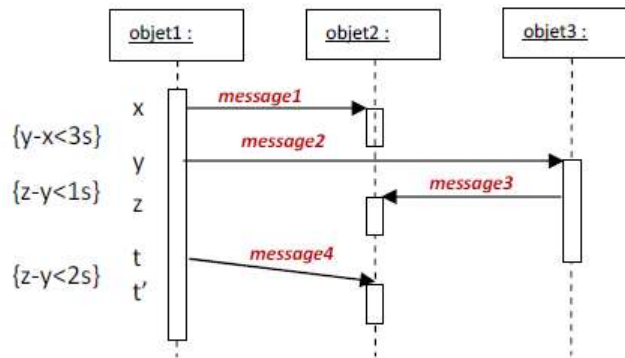
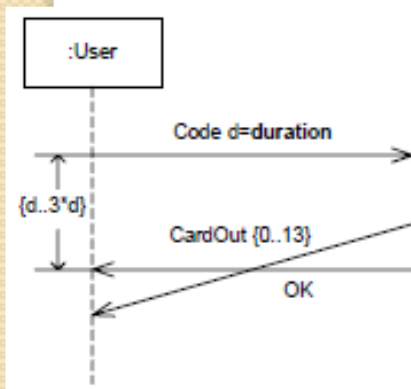


Gates



Timing constraints

- Time markers with constraints may be placed along the lifeline. A message with a non-negligible propagation time can be represented by an oblique arrow.



Active Object / Passive Object

- **Active Object**

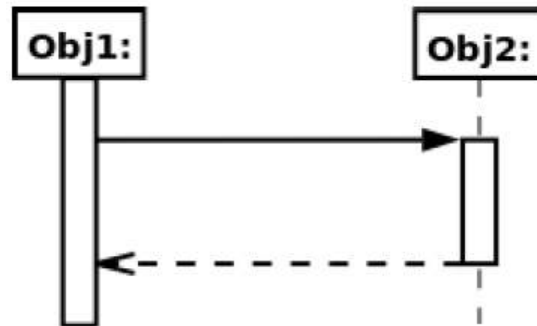
Initiates and controls the flow of activities.

The vertical dashed lifeline is replaced by a double vertical line.

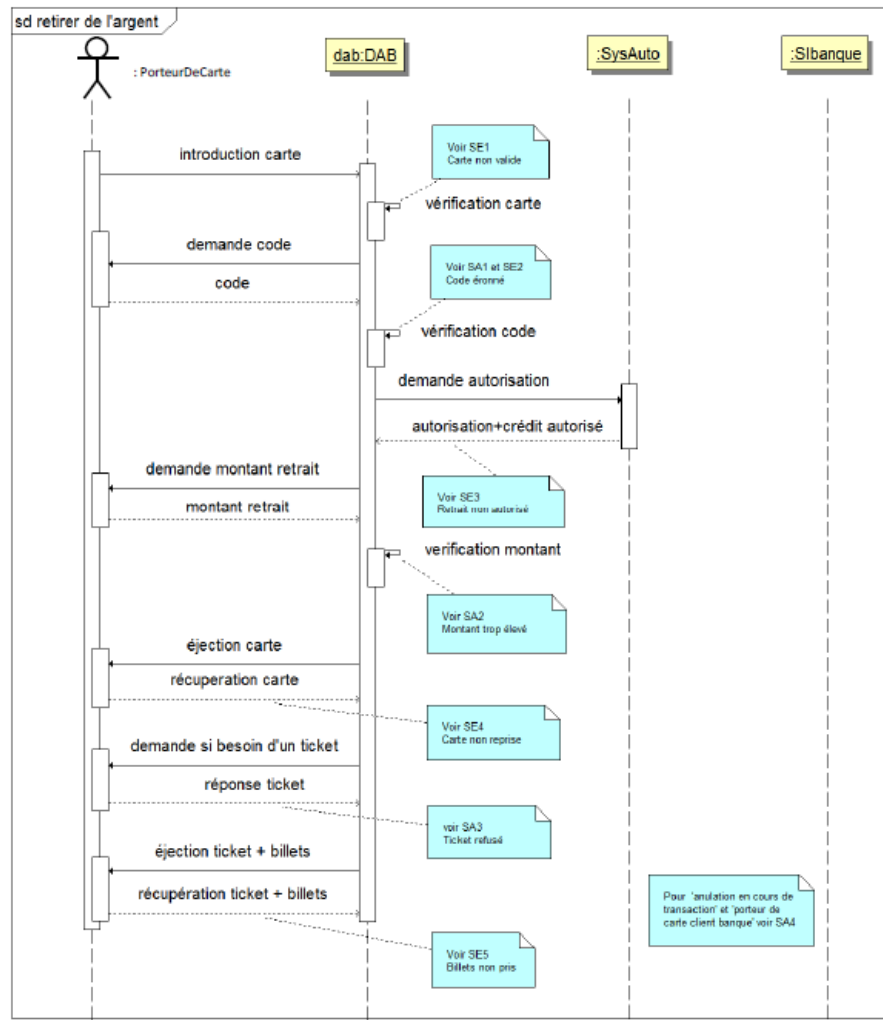
- **Passive Object**

Needs to be given the activity flow in order to execute a method.

The specification of the execution of a reaction on a passive object is represented by a white or gray rectangle placed on the dashed lifeline.



Example: withdraw money scenario



interaction fragment

An **interaction fragment** is a part of the sequence diagram (delimited by a rectangle) associated with a label (in the upper-left corner).

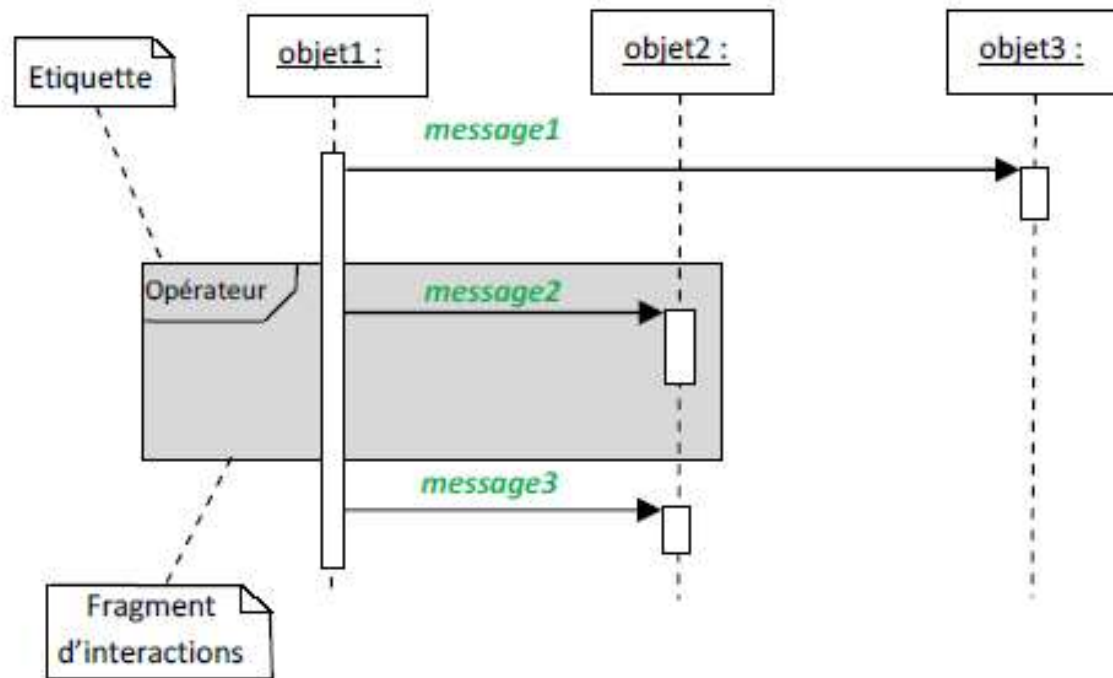
The label contains an **interaction operator** that allows describing the execution rules of the messages inside the frame.

The **operands** of an interaction operator are separated by a dashed line.

The **conditions for choosing operands** (if any) are given by Boolean expressions in brackets [].

The main execution modes include loops, conditional branches, alternatives, simultaneous message sending, etc.

An interaction fragment



Control flow operators

- **opt (optional)**: Contains a sequence that may or may not occur. In the guard, you can specify the condition under which it occurs.
- **alt (alternative)**: Contains a list of fragments in which different message sequences exist. Only one sequence can occur at a time.
- **loop**: The fragment is repeated a certain number of times. In the guard, the condition under which it should be repeated is specified.
- **break**: If this fragment is executed, the rest of the sequence is abandoned. You can use the guard to indicate the condition under which the break will occur.
- **par (parallel)**: The events in the fragments may be interleaved.
- **critical**: Used inside a *par* or *seq* fragment. Indicates that the messages in the fragment must not be interleaved with other messages.
- **seq (weak sequencing)**: There are at least two operand fragments. Messages involving the same lifeline must occur in the order of the fragments. When they do not involve the same lifelines, messages from different fragments may be interleaved in parallel.
- **strict**: There are at least two operand fragments. The fragments must occur in the given order.

Control flow operators

- **consider:** Specifies a list of messages that this fragment describes. Other messages may occur in the running system, but they are not relevant to the purpose of this description.
- **ignore:** Lists the messages that this fragment does not describe. They may occur in the running system, but they are not relevant to the purpose of this description.
- **assert:** The operand fragment specifies the only valid sequences. Usually used within a *consider* or *ignore* fragment.
- **neg:** The sequence shown in this fragment must not occur. Usually used within a *consider* or *ignore* fragment.

Sequence reuse

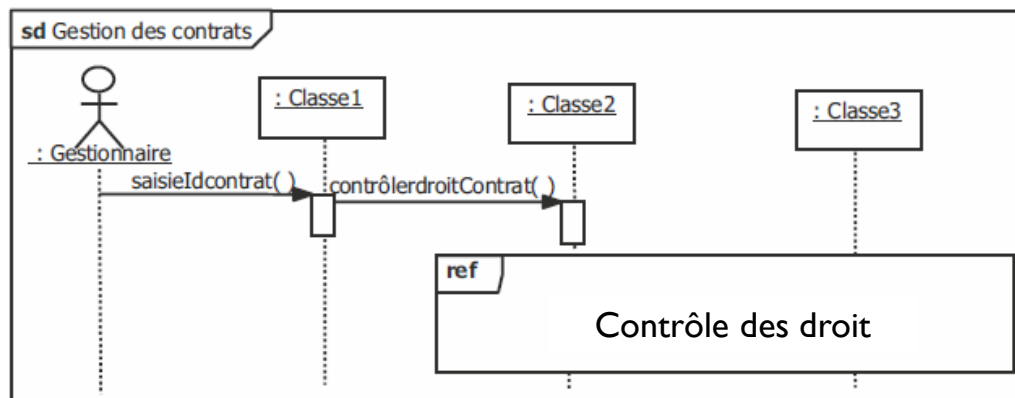
- **Ref (Reference):** Reuse of sequences
A *ref* fragment is used to indicate the reuse of a sequence diagram defined elsewhere.

Sequence reuse

- **Ref Operator:**

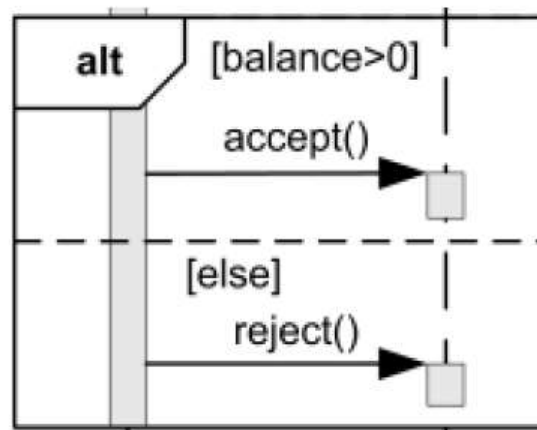
The *ref* operator is used to call an interaction sequence described elsewhere, thus acting as a kind of sequence sub-diagram.

In this example, a fragment called “Access Rights Control” is invoked, which is described elsewhere.



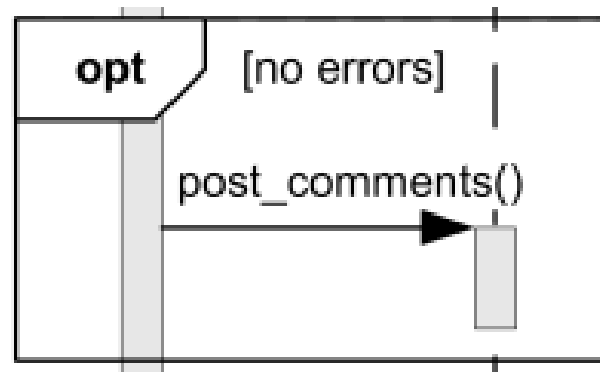
Alternative Operator(alt)

- **Combined Interaction Fragments – Alternative Operator**
The fragment represents a choice.
- Example: if "**balance > 0**" then it is **accepted**, otherwise it is **rejected**.



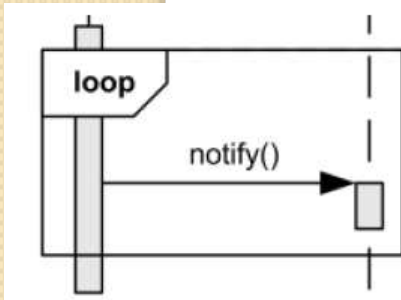
Option Operator (opt)

- **Combined Interaction Fragments – Option Operator**
Equivalent to an alternative operator with only one condition.
- Example: if "**no error**" then the object posts a comment.

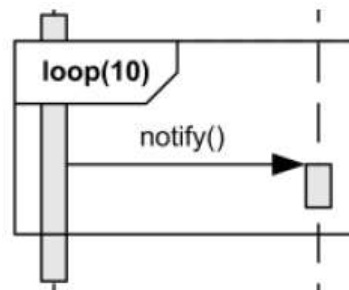


Loop Operator(loop)

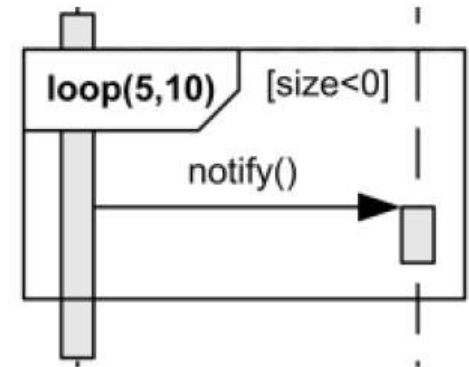
- **Combined Interaction Fragments – Loop Operator**
Used to specify a loop.



A loop that may possibly execute an infinite number of times.



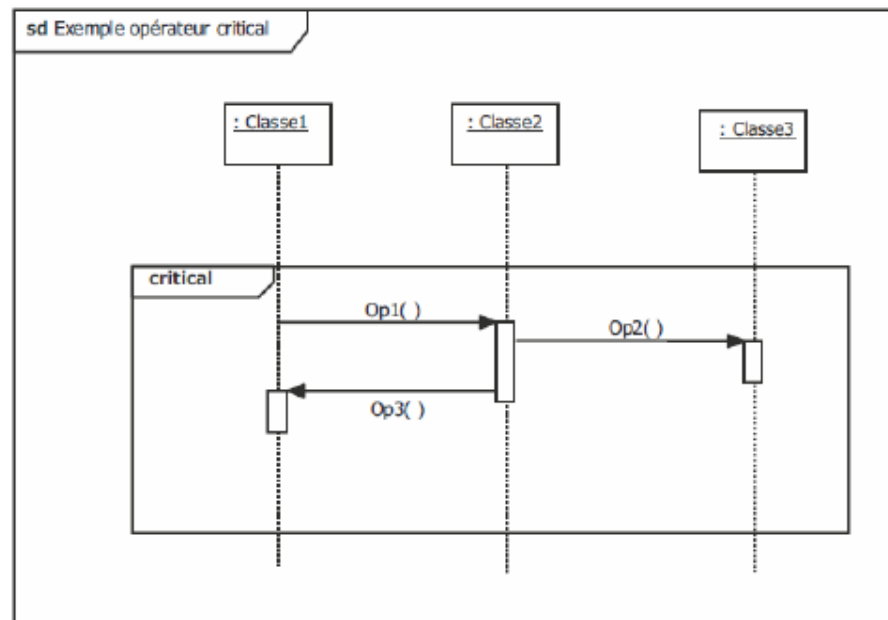
A loop that executes 10 times



The loop executes at least 5 times and at most 10 times. If the condition is false, the loop terminates, regardless of how many times it has already executed.

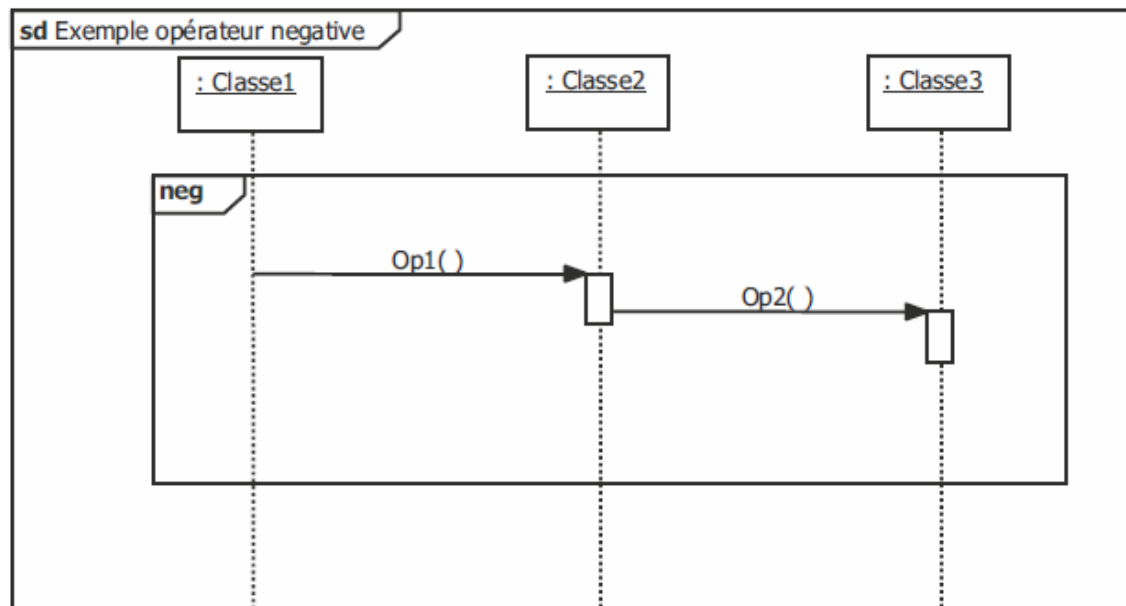
critical operator

The **critical operator** is used to indicate that a sequence of interactions cannot be interrupted due to the critical nature of the operations being processed.
The interactions contained within the critical sequence are considered atomic.



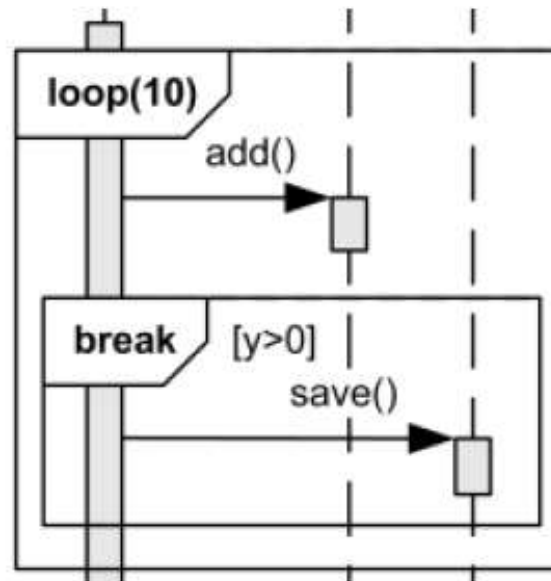
Negative Operator (neg)

- **Negative Operator:**
The *neg* (negative) operator is used to indicate that an interaction sequence is invalid.



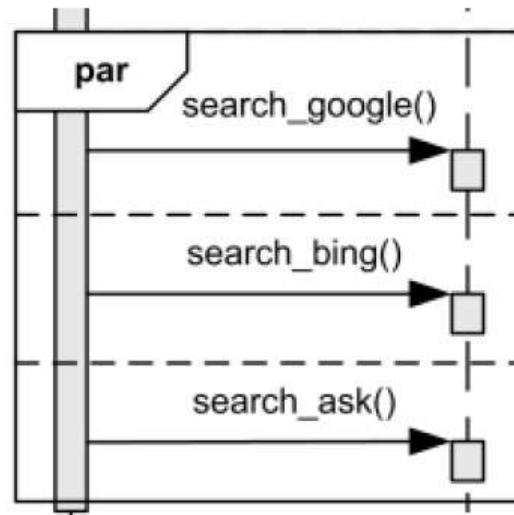
break

- If the **break** condition is true, the associated fragment is executed and the rest of the fragment in which the break is included is ignored. If the **break** condition is false, execution continues directly with the remainder of the enclosing fragment.



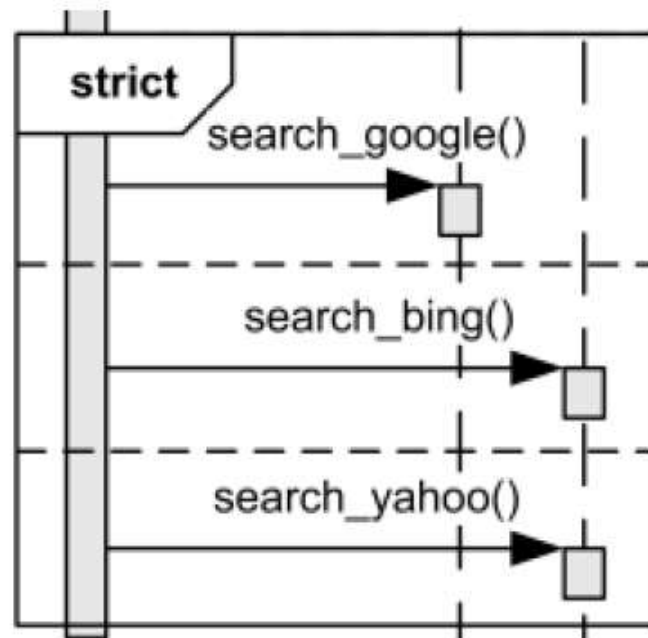
Parallel Operator (par)

- **Combined Interaction Fragments – Parallel Operator**
Specifies the parallel execution of several sub-fragments.



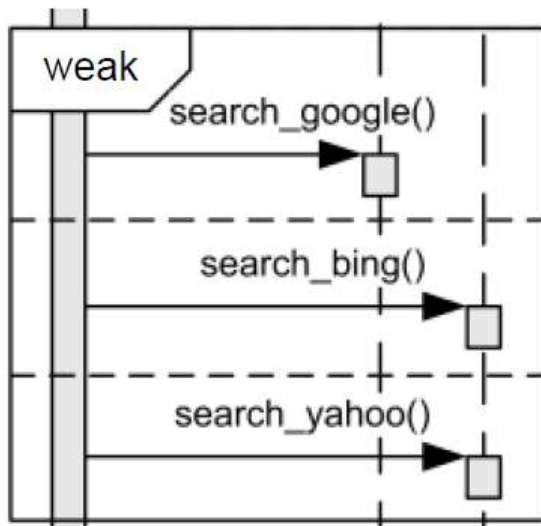
Strict Sequencing Operator

- **Combined Interaction Fragments – Strict Sequencing Operator**
The fragments execute one after another.



Weak Sequencing Operator

- **Combined Interaction Fragments – Weak Sequencing Operator**
- If the lifelines involved are the same → sequential execution
If the lifelines involved are different → parallel execution

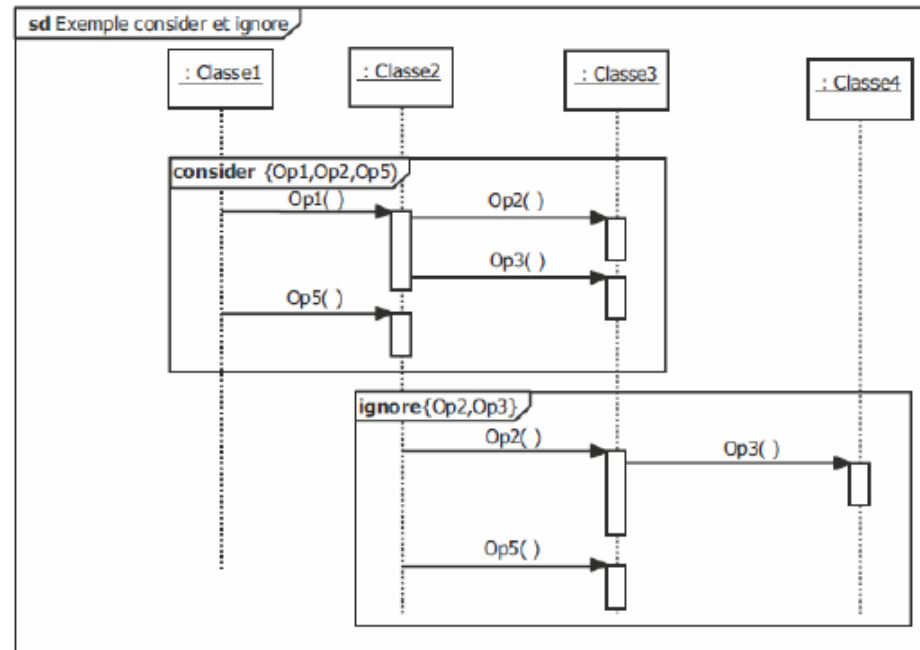


Google in parallel with Bing and/or Yahoo
Bing then Yahoo

Ignore and Consider Operators

- **Ignore and Consider Operators:**

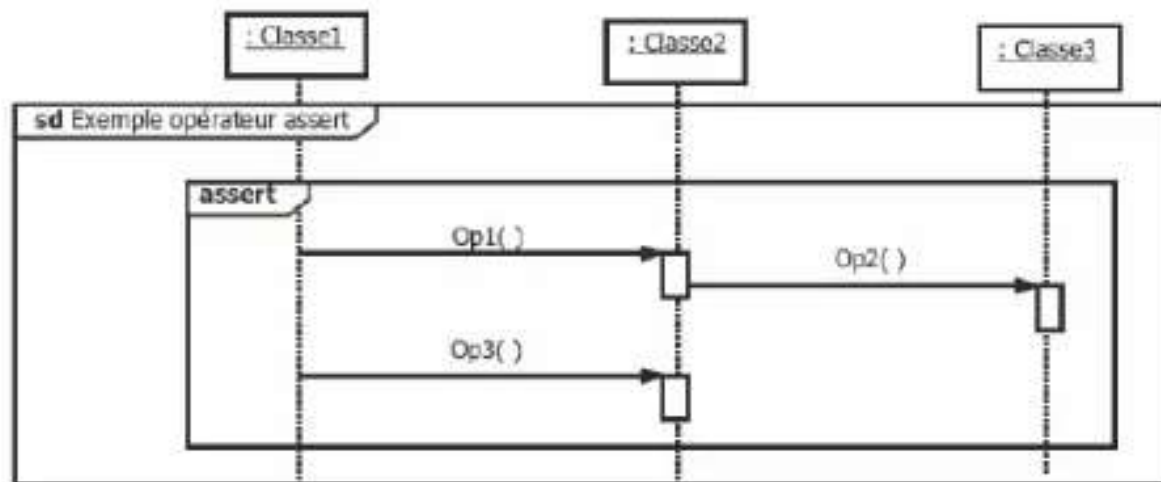
The *ignore* and *consider* operators are used for interaction fragments in which we want to show that certain messages may either be absent without affecting the course of interactions (*ignore*), or must necessarily be present (*consider*).



Assert

- operator defines a **combined fragment** that specifies a sequence of interactions which **must occur in every valid execution** of the system.

It represents a **mandatory behavior constraint** — if the actual system behavior deviates from the interactions described within the assert fragment, the **model is considered invalid**.



Continuation

- A Continuation is used in seq and alt Combined Fragments, to indicate the branches of continuation that an operand follows. To indicate a continuation, end an operand with a Continuation, and indicate the continuation branch with a matching Continuation (same name) preceding the Interaction Fragment.

